Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Унарные и бинарные операции над графами»

Выполнили студенты группы 25ВВВ2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Унарные и бинарные операции над графами

Цель работы

изучение унарных и бинарных операций над графами и реализация на языке Go операций отождествления вершин, стягивания ребра, расщепления вершины, объединения, пересечения, кольцевой суммы и декартова произведения графов для представлений в виде матрицы и списков смежности.

Лабораторное задание

1. Сгенерировать с помощью генератора случайных чисел две матрицы смежности M1, M2 неориентированных помеченных графов G1, G2 и вывести их на экран. * Преобразовать матрицы смежности в списки смежности и вывести полученные списки на экран.
2. Для матричной формы представления графов выполнить операции: а) отождествления вершин; б) стягивания ребра; в) расщепления вершины. Номера вершин вводятся с клавиатуры. * Выполнить те же операции для представления графов в виде списков смежности.
3. Для матричной формы представления графов выполнить операции: а) объединения $G = G1 \cup G2$; б) пересечения $G = G1 \cap G2$; в) кольцевой суммы $G = G1 \oplus G2$.
4.
   - Для матричной формы представления графов выполнить операцию декартова произведения графов $G = G1 \times G2$.

Ход работы

Граф хранится в структуре Graph, которая содержит срез меток вершин Names и матрицу смежности M. Для списков смежности используется структура ListGraph, в которой для каждой вершины хранится срез индексов смежных с ней вершин. Графы неориентированные, без петель, поэтому матрица

смежности симметрична, а на главной диагонали стоят нули. Вершины нумеруются с единицы (v1, v2, …), при вводе с клавиатуры номер приводится к индексу массива вычитанием единицы.

Программа управляется текстовым меню. В нём доступны генерация графов, вывод матриц и списков смежности, унарные операции над матрицами и над списками, операции объединения, пересечения и кольцевой суммы, а также декартово произведение. Ввод номеров вершин проверяется: значение должно быть целым числом из допустимого диапазона, при ошибке запрос повторяется.

Генерация графов и списки смежности

Функция randomGraph заполняет верхний треугольник матрицы случайными значениями: ребро между вершинами i и j создаётся с вероятностью 0,4, после чего значение копируется в симметричную ячейку. Размеры графов G1 и G2 задаёт пользователь. Функция toList просматривает каждую строку матрицы и для каждой единицы добавляет номер столбца в список соответствующей вершины.

```
func randomGraph(n int) *Graph {
    g := &Graph{Names: make([]string, n), M: newMatrix(n)}
    for i := 0; i < n; i++ {
        g.Names[i] = fmt.Sprintf("v%d", i+1)
    }
    for i := 0; i < n; i++ {
        for j := i + 1; j < n; j++ {
            if rand.Float64() < 0.4 {
                g.M[i][j] = 1
                g.M[j][i] = 1
            }
        }
    }
    return g
}
```

Отождествление вершин и стягивание ребра

При отождествлении вершин u и v из графа удаляются обе вершины, добавляется новая вершина z, и она соединяется с каждой вершиной окружения

Q = Q1 ∪ Q2, то есть с каждой вершиной, смежной хотя бы с одной из u и v. Сами вершины u и v в окружение не входят. Стягивание ребра является отождествлением смежных вершин, поэтому функция contractMatrix перед вызовом identifyMatrix проверяет, что ребро (u, v) существует; если ребра нет, выводится сообщение об ошибке.

```
func identifyMatrix(g *Graph, u, v int) *Graph {
    n := len(g.M)
    keep := []int{}
    for i := 0; i < n; i++ {
        if i != u && i != v {
            keep = append(keep, i)
        }
    }
    k := len(keep)
    res := &Graph{Names: make([]string, k+1), M: newMatrix(k + 1)}
    for a, i := range keep {
        res.Names[a] = g.Names[i]
        for b, j := range keep {
            res.M[a][b] = g.M[i][j]
        }
        if g.M[u][i] == 1 || g.M[v][i] == 1 {
            res.M[a][k] = 1
            res.M[k][a] = 1
        }
    }
    res.Names[k] = "z"
    return res
}
```

Для списков смежности реализованы аналогичные функции identifyList и contractList. В них сначала строится множество Q из соседей вершин u и v, затем для оставшихся вершин пересчитываются индексы, из их списков удаляются u и v, а в списки вершин из Q добавляется новая вершина z.

Расщепление вершины

При расщеплении вершина v заменяется двумя вершинами v’ и v’‘, которые соединяются между собой ребром. Соседи исходной вершины распределяются между ними поочерёдно: первый сосед остаётся у v’, второй переходит к v’‘, третий остаётся у v’ и так далее. Способ распределения в методических указаниях не задан, поэтому выбран такой, при котором операция является обратной к стягиванию ребра (v’, v’’).

```
func splitMatrix(g *Graph, v int) *Graph {
    n := len(g.M)
    res := &Graph{Names: make([]string, n+1), M: newMatrix(n + 1)}
    for i := 0; i < n; i++ {
        res.Names[i] = g.Names[i]
        for j := 0; j < n; j++ {
            res.M[i][j] = g.M[i][j]
        }
    }
    res.Names[v] = g.Names[v] + "'"
    res.Names[n] = g.Names[v] + "''"
    cnt := 0
    for w := 0; w < n; w++ {
        if g.M[v][w] == 1 {
            if cnt%2 == 1 {
                res.M[v][w], res.M[w][v] = 0, 0
                res.M[n][w], res.M[w][n] = 1, 1
            }
            cnt++
        }
    }
    res.M[v][n], res.M[n][v] = 1, 1
    return res
}
```

Функция splitList выполняет то же преобразование над списками смежности: у переносимых соседей номер v в списке заменяется на номер новой вершины, а списки вершин v’ и v’’ дополняются ссылкой друг на друга.

Объединение, пересечение и кольцевая сумма

В бинарных операциях вершины с одинаковыми номерами считаются одной и той же вершиной, поэтому графы разного размера сравниваются по общим номерам. Объединение определено на множестве вершин V1 ∪ V2: берётся большее число вершин, и ребро включается в результат, если оно есть хотя бы в одном графе. Пересечение определено на V1 ∩ V2: берётся меньшее число вершин, и ребро включается, если оно есть в обоих графах. Для кольцевой суммы ребро включается, если оно есть ровно в одном из графов (операция исключающего ИЛИ над элементами матриц). После этого вершины, оставшиеся без рёбер, удаляются, так как кольцевая сумма не содержит изолированных вершин.

```
func ringSum(a, b *Graph) *Graph {
    n := max(len(a.M), len(b.M))
    full := newMatrix(n)
    get := func(g *Graph, i, j int) int {
```

```
        if i < len(g.M) && j < len(g.M) {
            return g.M[i][j]
        }
        return 0
    }
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            full[i][j] = get(a, i, j) ^ get(b, i, j)
        }
    }
    // ... удаление изолированных вершин и переименование оставшихся
}
```

Декартово произведение

Множество вершин произведения G1 × G2 есть декартово произведение множеств вершин, поэтому в результате получается n1·n2 вершин. Вершине (i, k) соответствует индекс i·n2 + k. Вершины (zi, vk) и (zj, vl) смежны тогда и только тогда, когда i = j и вершины vk, vl смежны в G2, либо k = l и вершины zi, zj смежны в G1.

```
func cartesianProduct(a, b *Graph) *Graph {
    n1, n2 := len(a.M), len(b.M)
    n := n1 * n2
    res := &Graph{Names: make([]string, n), M: newMatrix(n)}
    for i := 0; i < n1; i++ {
        for k := 0; k < n2; k++ {
            res.Names[i*n2+k] = fmt.Sprintf("(%d,%d)", i+1, k+1)
        }
    }
    for i := 0; i < n1; i++ {
        for k := 0; k < n2; k++ {
            for j := 0; j < n1; j++ {
                for l := 0; l < n2; l++ {
                    if (i == j && b.M[k][l] == 1) || (k == l && a.M[i][j] == 1)
{
                        res.M[i*n2+k][j*n2+l] = 1
                    }
                }
            }
        }
    }
    return res
}
```

Из-за быстрого роста размера результата программа отказывается выводить произведение, если оно содержит более 100 вершин.

Оценка сложности операций

Пусть n — число вершин графа, m — число рёбер. Преобразование матрицы в списки смежности требует просмотра всех элементов матрицы, то есть $O(n^2)$. Для матричного представления отождествление, стягивание и расщепление строят новую матрицу и поэтому выполняются за $O(n^2)$ по времени и памяти. Для списков смежности те же операции выполняются за время, пропорциональное размеру графа, то есть $O(n + m)$ с учётом сортировки списков; проверка наличия ребра при стягивании занимает $O(\deg u)$. Объединение, пересечение и кольцевая сумма просматривают все элементы матриц и имеют сложность $O(n^2)$. Декартово произведение строится четырьмя вложенными циклами за $O(n1^2 \cdot n2^2)$, что равно размеру матрицы результата, поэтому его сложность и расход памяти быстро растут с увеличением размеров исходных графов.

Результаты работы программы

Ниже приведены примеры работы операций на графах G1 (цикл из четырёх вершин) и G2 (три вершины, рёбра v1–v2 и v1–v3).

Матрицы смежности исходных графов:

```
G1:                     G2:
    v1 v2 v3 v4             v1 v2 v3
v1   0  1  0  1         v1   0  1  1
v2   1  0  1  0         v2   1  0  0
v3   0  1  0  1         v3   1  0  0
v4   1  0  1  0
```

Рисунок 1 — Сгенерированные матрицы смежности G1 и G2

Списки смежности, полученные из матриц:

```
G1:                     G2:
v1: v2, v4              v1: v2, v3
v2: v1, v3              v2: v1
v3: v2, v4              v3: v1
v4: v1, v3
```

Рисунок 2 — Списки смежности графов G1 и G2

Унарные операции над графом G1. Отождествление несмежных вершин v1 и v3: вершины удалены, новая вершина z соединена с v2 и v4, так как это окружение обеих отождествляемых вершин.

```
    v2 v4  z
v2   0  0  1
v4   0  0  1
z    1  1  0
```

Рисунок 3 — Отождествление вершин v1 и v3 (матрица смежности)

Стягивание ребра (v1, v2): окружением вершины z являются v3 и v4, ребро (v3, v4) сохраняется, поэтому получается треугольник.

```
    v3 v4  z
v3   0  1  1
v4   1  0  1
z    1  1  0
```

Рисунок 4 — Стягивание ребра (v1, v2) (матрица смежности)

Расщепление вершины v1: соседи v2 и v4 распределены между вершинами v1’ и v1’’, которые соединены ребром. Получился цикл из пяти вершин.

```
     v1' v2 v3 v4 v1''
v1'    0  1  0  0   1
v2     1  0  1  0   0
v3     0  1  0  1   0
v4     0  0  1  0   1
v1''   1  0  0  1   0
```

Рисунок 5 — Расщепление вершины v1 (матрица смежности)

Те же операции для списков смежности дают такие же графы:

```
Отождествление v1, v3:   Стягивание (v1, v2):   Расщепление v1:
v2: z                    v3: v4, z              v1':  v2, v1''
v4: z                    v4: v3, z              v2:   v1', v3
z:  v2, v4               z:  v3, v4             v3:   v2, v4
                                                v4:   v3, v1''
                                                v1'': v1', v4
```

Рисунок 6 — Результаты унарных операций для списков смежности

Бинарные операции над графами G1 и G2. В объединении 4 вершины и рёбра v1–v2, v1–v3, v1–v4, v2–v3, v3–v4.

```
    v1 v2 v3 v4
v1   0  1  1  1
v2   1  0  1  0
v3   1  1  0  1
v4   1  0  1  0
```

Рисунок 7 — Объединение G1 ∪ G2

В пересечении 3 вершины (общие для обоих графов) и единственное общее ребро v1–v2.

	v1	v2	v3
v1	0	1	0
v2	1	0	0
v3	0	0	0

Рисунок 8 — Пересечение G1 ∩ G2

В кольцевую сумму входят рёбра, присутствующие ровно в одном графе: v1–v3, v1–v4, v2–v3, v3–v4. Общее ребро v1–v2 исключено, изолированных вершин нет.

	v1	v2	v3	v4
v1	0	0	1	1
v2	0	0	1	0
v3	1	1	0	1
v4	1	0	1	0

Рисунок 9 — Кольцевая сумма G1 ⊕G2

Декартово произведение. Для наглядности взяты граф с двумя вершинами и ребром z1–z2 и граф-цепочка v1–v2–v3; результат представляет собой «лестницу» из шести вершин.

	(1,1)	(1,2)	(1,3)	(2,1)	(2,2)	(2,3)
(1,1)	0	1	0	1	0	0
(1,2)	1	0	1	0	1	0
(1,3)	0	1	0	0	0	1
(2,1)	1	0	0	0	1	0
(2,2)	0	1	0	1	0	1
(2,3)	0	0	1	0	1	0

Рисунок 10 — Декартово произведение графов

Во всех случаях результаты, выданные программой, совпали с ожидаемыми.

Вывод

В ходе выполнения лабораторной работы были изучены унарные и бинарные операции над графами, и на языке Go реализованы отождествление вершин, стягивание ребра, расщепление вершины, объединение, пересечение, кольцевая сумма и декартово произведение. Унарные операции выполнены как для матричного представления, так и для списков смежности. Анализ показал, что над матрицей смежности операции выполняются за $O(n^2)$, над списками

смежности унарные операции выполняются за время, пропорциональное размеру графа, а декартово произведение требует $O(n1^2 \cdot n2^2)$ операций. Результаты работы программы совпали с ожидаемыми, что подтверждает корректность реализации.